

ECE 6913 Computing System Architecture

CSA Project Phase 1 Report

Yixuan Zhang yz6909

Oct 27, 2024

1. Code running

The file to run is *main.cpp* in folder *code*. Programming language used in this project is C++. There are additional libraries and packages included before running the file.

```
#include <iostream>
#include <stdio.h>
#include <string>
#include <fstream>
#include <sstream>
#include <vector>
```

In order to run the file, first need to compile it use `g++ -o my_program main.cpp`. The keyword *my_program* is the name of executable file which can be substitute by other names. Then use `./my_program` to execute, where the file requires the user to enter the path to *imem.txt* and *dmem.txt*. The form of the path should be `/users/the/path/to/file` which is different from path using “\\” in skeleton code. After execution, all result files will be output to the local file, the folder contains *imem.txt* and *dmem.txt*.

2. Project tasks

2.1 Draw the schematic for a single stage core and fill in your code in the to run the simulator.

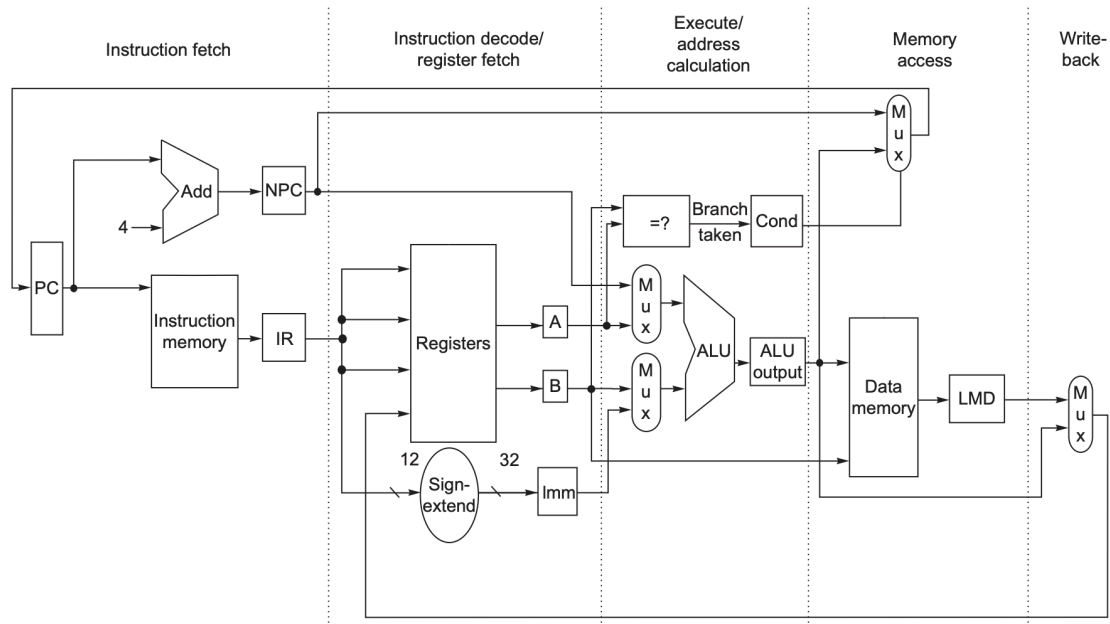


Fig. 1 Single Stage Core schematic

Implementation: In SSP, IF stage reads an instruction from instruction memory using current PC. ID stage decodes the instruction to get funct3, rs, rt, read_data1, read_data2, Immediate, Wrt_reg_addr, alucontrol, rd_mem, wrt_mem, wrt_enable, is_I_type information. The Arithmetic Logic Unit is the core of EXE Stage which finish add, sub, and, or, xor functions between read_data1 and (read_data2 or immediate). The ALU result will be used to store in RF, store in Memory, be the offset, be the determination of branch and jump instructions. The branch function make the next PC to designated one, not simply +4. Where to use ALU result is determined by Multiplexer, implemented through transmission of rd_mem, wrt_mem, wrt_enable signals. MEM stage allows the core to write to or read from data memory if rd_mem, wrt_mem signal is assigned to true. WB stage allows Wrt_data to be written into RF, from ALU result or data read from data memory. “nop” signal transits stage by stage with specific value assigned to when there is specific instruction (for example, branch action set nextstate.IF.nop to true).

2.2 Draw the schematic for a five stage pipelined core and fill in your code to run the simulator. The core should be able to take care of RAW and control hazards by stalling and forwarding.

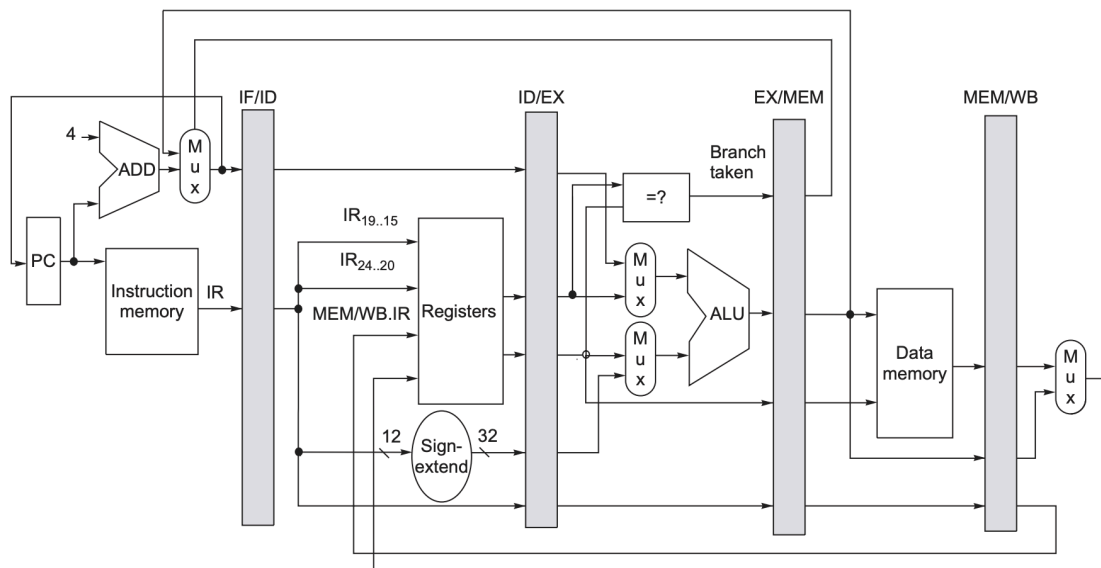


Fig. 2 Pipelined Five Stage Core schematic

The running logic is similar to SSP with five stages running in reversed order. The reversed order allows the stages of different instructions can be executed in the same cycle. The signals of current state will be used first before changed by a previous stage. All changing value is stored to nextstate so that all signals will be updated in a new cycle. There will be RAW hazards and branch hazards during the execution of FSP. The solution is to do forwarding when wrt_reg_address has collisions with rs or rt and discard next instruction when a branch action should be taken. By using forwarding instead of stalling, fewer amount of cycles will be used, therefore improve execution time.

2.3 Measure and report average CPI, Total execution cycles, and Instructions per cycle for both these cores by adding performance monitors to your code.

```
Single Stage Core Performance Metrics
Number of cycles taken: 6
Cycles per instruction: 1.200000
Instructions per cycle: 0.833333
```

Fig. 3 Performance Metrics of SSP in testcase1

```
Five Stage Core Performance Metrics
Number of cycles taken: 10
Cycles per instruction: 2.000000
Instructions per cycle: 0.500000
```

Fig. 4 Performance Metrics of FSP in testcase1

```
Single Stage Core Performance Metrics
Number of cycles taken: 40
Cycles per instruction: 1.025641
Instructions per cycle: 0.975000
```

Fig. 5 Performance Metrics of SSP in testcase2

```
Five Stage Core Performance Metrics
Number of cycles taken: 44
Cycles per instruction: 1.128205
Instructions per cycle: 0.886364
```

Fig. 6 Performance Metrics of FSP in testcase2

```
Single Stage Core Performance Metrics
Number of cycles taken: 7
Cycles per instruction: 1.166667
Instructions per cycle: 0.857143
```

Fig. 7 Performance Metrics of SSP in testcase3

Five Stage Core Performance Metrics
Number of cycles taken: 12
Cycles per instruction: 2.000000
Instructions per cycle: 0.500000

Fig. 8 Performance Metrics of FSP in testcase3

The above are performance metrics for testcase1, testcase2, testcase3 for both Single Stage Core and Five Stage Core. The output file can be seen in submission folder.

2.4 Compare the results from both the single stage and the five stage pipelined processor implementations and explain why one is better than the other.

Metrics	Single Stage Core	Five Stage Core
CPI	Low	High
IPC	High	Low
Execution time	High	Low
Hazards	No hazards	RAW hazards, branch hazards
Pipeline	Not pipelined	Pipelined

SSC has lower CPI and higher IPC than FSC. But it has higher execution time because it executes one instruction each cycle without hazards, leading to higher cycle time than FSC, with higher clock rate and throughput.

By dealing with hazards in FSC using forwarding or stalling, FSC can have better performance in execution, which is better than SSC.

2.5 What optimizations or features can be added to improve performance?

Using forwarding instead of stalling can reduce the cycles executed in FSC.

Do branch prediction to eliminate instructions discarded.

Allowing parallel execution of independent instructions in multiple pipelines, improving IPC.